

Design a News Feed

Twitter · Instagram · LinkedIn · Facebook – "post పెట్టగానే followers అందరి inbox lo రాసేద్దాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – ఒక్క celebrity post మిగతా అందరి posts ని **3.3 నిమిషాలు** ఆపేస్తుంది. రెండోది – దానికి సహజమైన పరిష్కారం DB queries ని **52 రెట్లు** పెంచి, తెచ్చిన దాంట్లో **3.8% మాత్రమే** వాడుతుంది. మూడోది – inbox lo రాసిన దాంట్లో **90% ఎవరూ చదవరు**. మరియు చివర్లో ఒక నాలుగోది: ఈ hybrid ని సరిగ్గా రాయడానికి నాకు **ఏడు ప్రయత్నాలు** పట్టింది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` `lo` run చేసినదే.

HLD_DESIGN_PROBLEMS §3 ఈ PROBLEM ని "SYSTEM DESIGN యొక్క గుండె" అంటుంది – ఎందుకే ఇక్కడ కొలుస్తాం

ఆ doc lo **fan-out on write vs read** అనే trade-off ని ఒక పట్టికలో ఇచ్చాను, మరియు అది సరైనదే. "Celebrity problem" గురించీ చెప్పాను.

కానీ ఒక పట్టిక చెప్పలేనిది ఒకటి ఉంది: ఈ trade-off యొక్క రెండు వైపులా ఎంత లోతు ఉంది.

§4 lo ఒక celebrity post మిగతా అందరి posts ని ఎంతసేపు ఆపుతుందో కొలుస్తాం – **56 ms నుంచి 199.9 సెకన్లు**. §7 lo "అయితే pull చేద్దాం" అనే పరిష్కారం ఎంత ఖరీదే – **52 రెట్లు queries, 96% వృథా**.

మరియు ఒక విషయం ఆ పట్టికలో అస్సలు లేదు, మరియు అదే ఈ doc యొక్క అసలు కథ:

"Hybrid వాడండి" అనేది ఒక జవాబు కాదు – అది ఒక కొత్త problem యొక్క పేరు.

§13 lo ఆ hybrid ని సరిగ్గా రాయడానికి నేను **ఎడుసార్లు** ప్రయత్నించాను. ప్రతిసారీ ఒక రంధ్రం మూసి, ఇంకొకటి తెరిచాను. ఆ ఎడు ప్రయత్నాల జాబితా ఈ doc lo అత్యంత విలువైన భాగం – ఎందుకంటే interview lo మిమ్మల్ని అడిగేది సరిగ్గా అదే: **"మీ hybrid ఎక్కడ తప్పు అవుతుంది?"**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు followers ఎలా పంచబడతారు?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ఒక్క celebrity

3. Step – post పెట్టగానే అందరి inbox lo రాయడం
4. మొదటి విరుపు – ఒక post, అందరికీ 3.3 నిమిషాల ఆలస్యం

Part 3 – రెండో విరుపు: అయితే చదివేటప్పుడు కలుపుదాం

5. Step – fan-out on read
6. రెండో విరుపు – 52 రెట్లు queries, 96% వృథా

Part 4 – మూడో విరుపు: ఎవరూ చదవని inbox

7. Step – hybrid · celebrities ని pull, మిగతా అందరినీ push
8. మూడో విరుపు – రాసిన దాంట్లో 90% చదవబడదు

Part 5 – పూర్తి system

9. Step – inbox ఒక cache, ఒక సత్యం కాదు
10. మొత్తం code · ఏడు ప్రయత్నాలు · 2 లక్షల timelines

Part 6 – Interview lo

11. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
12. నోటీస్ చెప్పాల్సిన English script
13. Follow-ups – ranking, stories, unfollow
14. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు followers ఎలా పంచబడతారు?

ఒక user post పెడతాడు. అతని followers అందరి timeline lo అది కనిపించాలి. Timeline అంటే – నేను follow చేసే వాళ్ళందరి posts, **కొత్తవి ముందు**.

ఇది సులభంగా అనిపిస్తుంది. రెండు స్పష్టమైన దారులు ఉన్నాయి:

- **Fan-out on write** – post పెట్టగానే followers అందరి "inbox" lo ఒక ప్రతి రాయడం. చదవడం వేగం, రాయడం ఖరీదు.
- **Fan-out on read** – ఎక్కడా రాయకుండా, timeline అడిగినప్పుడు followees అందరి posts తెచ్చి కలపడం. రాయడం వేగం, చదవడం ఖరీదు.

ఏది ఎంచుకోవాలో నిర్ణయించే **ఒకే ఒక్క సంఖ్య** ఉంది, మరియు అది followers సంఖ్య కాదు – **followers యొక్క పంపిణీ**.

2,00,000 users · followers పంపిణీ

మధ్యస్థం (p50)	: 93
p90	: 397
p99	: 3,057
p99.9	: 24,890
గరిష్టం	: 19,03,356
సగటు	: 349

మధ్యస్థం 93, గరిష్ఠం 19 లక్షలు – 20,000 రెట్లు తేడా

మరియు సగటు (349) మధ్యస్థం (93) కంటే నాలుగు రెట్లు ఎక్కువ. అదే power law యొక్క సంతకం: సగటు ఒక అబద్ధం.

ఈ ఒక్క ఆకారం నుంచి ఈ doc మొత్తం వస్తుంది. మీరు "సగటు post కి 349 writes" అని design చేస్తే, ఆ 349 ఎప్పుడూ జరగదు – ఒకసారి 93, ఇంకోసారి 19 లక్షలు.

ఆ తోక ఎంత బరువైనదో ఇలా చూడండి:

ప్రతి user ఒక post పెడితే – fan-out on write lo ఎన్ని writes?

మొత్తం writes : 6,98,23,594

సగటు : 349 writes / post

పైన 1% users సృష్టించే writes : 3,31,54,744 (47.5%)

పైన 0.1% users సృష్టించేవి : 2,06,22,481 (29.5%)

పైన 0.1% users మొత్తం పనిలో 29.5% సృష్టిస్తారు. 200 మంది, 7 కోట్ల writes lo 2 కోట్లు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Timeline కాలక్రమంలో నా, ranked నా?	Ranked అయితే ఇది ఒక ML problem కూడా. కాలక్రమం అనుకుంటే scope స్పష్టం – §13
ఒక post కనిపించడానికి ఎంత ఆలస్యం ఫరవాలేదు?	ఇదే fan-out on write ని సాధ్యం చేసే సంఖ్య. §4 lo అది 199.9 సెకన్లు అవుతుంది
అత్యధిక followers ఎంత?	ఈ ఒక్క సంఖ్య మీ design ని నిర్ణయిస్తుంది. 10,000 అయితే fan-out on write చాలు; 1 కోటి అయితే కాదు
రోజూ ఎంతమంది నిజంగా వస్తారు (DAU)?	§8 – DAU తక్కువైతే fan-out on write lo 90% వృథా
Timeline ఎంత లోతు చూపించాలి?	ఇది inbox పరిమాణాన్ని నిర్ణయిస్తుంది, మరియు §10 lo ఒక bug కి కారణమైంది
Unfollow చేస్తే పాత posts పోవాలా?	అవును అంటే inbox ని శుభ్రం చేయాలా, లేక చదివేటప్పుడు వడపోయాలా? §10
Post delete అయితే?	Inbox lo కోట్ల ప్రతులు ఉన్నాయి – వాటిని తీసేయాలా, లేక చదివేటప్పుడు వడపోయాలా?

ఒక ప్రశ్న అడిగితే మరు వేరుగా కనిపిస్తారు

"అత్యధిక followers ఉన్న వ్యక్తికి ఎంతమంది ఉన్నారు?"

ఎందుకంటే ఈ problem lo జవాబు సగటు నుంచి రాదు, గరిష్ఠం నుంచి వస్తుంది. \$4 lo చూస్తారు – 10,000 followers ఉన్న వాడు ఏమీ చెయ్యలేడు, కానీ 1 కోటి ఉన్నవాడు మొత్తం system ని 3 నిమిషాలు ఆపేస్తాడు.

చాలామంది "సగటు user కి ఎంతమంది followers?" అని అడుగుతారు. అది తప్పు ప్రశ్న.

Part 2 – మొదటి విరుపు: ఒక్క celebrity

3. Step – post పెట్టగానే అందరి inbox lo రాయడం

సహజమైన మొదటి design. ప్రతి user కి ఒక **inbox** – అతని timeline, ముందే తయారుగా:

```
function post(author, text) {
  const id = savePost(author, text);
  for (const follower of followersOf(author)) {
    inbox(follower).push(id);      // ← ప్రతి ఒక్కరికీ ఒక ప్రతి
  }
}

function timeline(user, limit) {
  return inbox(user).slice(-limit).reverse(); // ఒక్క read · ఇప్పటికే క్రమంలో
}
```

చదవడం **ఒక్క query**, ఎంతమందిని follow చేసినా. అది గొప్ప విషయం – ఎందుకంటే ఈ system 100:1 read-heavy.

రాయడం ఖరీదు, కానీ §1 పట్టిక ప్రకారం సగటు post కి 349 writes. సెకనుకి 50,000 writes చేయగలిగితే అది **7 మిల్లీసెకన్లు**. ఫరవాలేదు.

సగటు.

4. మొదటి విరుపు – ఒక post, అందరికీ 3.3 నిమిషాల ఆలస్యం

Fan-out ఒక **queue**. Posts వరుసగా దానిలోకి వెళ్తాయి, workers వాటిని తీసుకుని inbox writes చేస్తారు.

ఇప్పుడు ఆ queue lo ఒక celebrity post వస్తే? దాని వెనక ఉన్నవాళ్ళు **ఆగాల్సిందే**.

కొలుద్దాం. 2,000 సాధారణ posts, ప్రతి 10 ms కి ఒకటి. 10వ సెకనులో ఒక celebrity post చేస్తాడు:

fan-out: సెకనుకి 50,000 writes · ఒక celebrity 10వ సెకనులో post చేస్తాడు

celebrity followers	ఆ post కి పట్టిన సమయం	వెనక ఉన్న 100 posts సగటు ఆలస్యం	గరిష్ఠం
10,000	485 ms	154 ms	485 ms
1,00,000	2.3 s	1.9 s	2.3 s
10,00,000	20.3 s	19.9 s	20.3 s
1,00,00,000	200.3 s	199.9 s	200.3 s

(celebrity లేకపోతే వెనక ఉన్న posts సగటు ఆలస్యం: 56 ms)

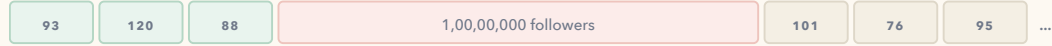
చివరి వరుస - ఒక్క POST వల్ల మిగతా అందరూ 3.3 నిమిషాలు ఆగారు

Celebrity లేనప్పుడు ఒక post కనిపించడానికి **56 ms**. ఒక కోటి followers ఉన్నవాడు post చేసిన తర్వాత - అతని వెనక వచ్చిన 100 posts సగటున **199.9 సెకన్లు** ఆగాయి.

ఆ 100 posts పెట్టినవాళ్ళలో ఎవరికీ celebrity తో సంబంధం లేదు. వాళ్ళకి **ఒక్కొక్కరికీ 93 followers** ఉండొచ్చు. వాళ్ళ post 2 milliseconds పని. కానీ వాళ్ళు **3.3 నిమిషాలు** ఆగారు.

మరియు celebrity యొక్క సొంత post - **చివరి follower దాన్ని 200 సెకన్ల తర్వాత** చూస్తాడు. అదే breaking news అయితే అది పనికిరానిది.

ఒక queue - ఒక పిడ్ల పని - వెనక అందరూ



↓ వాళ్ళు 200 సెకన్లు ఆగుతారు

ఒక post కనిపించడానికి వట్టి సమయం

56 ms - celebrity లేకపోతే

1.9 s - 1 లక్ష followers

199.9 s - 1 కోటి followers

ఇది celebrity యొక్క సమస్య కాదు - ఇది అతని వెనక ఉన్న అందరి సమస్య.
ఒక్క పని మొత్తం system ని ఆపేసింది. అదే head-of-line blocking.

మరియు ఇది ఎప్పుడూ కనిపించదు

§1 lo చూశాం - 0.1% users 29.5% పని సృష్టిస్తారు. అంటే celebrity posts **అరుదు**.

నేను 2,000 posts ని యాదృచ్ఛికంగా నమూనా తీసినప్పుడు, వాటిలో ఒక్క celebrity కూడా రాలేదు - కాబట్టి p50 మరియు p99 రెండూ **ఆరోగ్యంగా** కనిపించాయి.

మీ dashboard lo ఈ సమస్య రోజంతా కనిపించదు. అది రోజుకి రెండు మూడు సార్లు కనిపిస్తుంది, ప్రతిసారీ 3 నిమిషాలు - మరియు అవే మీ అత్యంత రద్దీ క్షణాలు.

Part 3 – రెండో విరుపు: అయితే చదివేటప్పుడు కలుపుదాం

5. Step – fan-out on read

సహజమైన వ్యతిరేక దారి: ఎక్కడా రాయకూడదు. Post ని ఒకే చోట నిల్వ చేసి, timeline అడిగినప్పుడు కలపడం:

```
function post(author, text) {
  savePost(author, text);          // ఒక్క write • celebrity అయినా అంతే
}

function timeline(user, limit) {
  const all = [];
  for (const a of following(user)) {
    all.push(...recentPostsOf(a, limit)); // ప్రతి followee నుంచి
  }
  return all.sort(byTimeDesc).slice(0, limit);
}
```

Celebrity problem పూర్తిగా పోయింది – 1 కోటి followers ఉన్నా post చేయడం ఒక్క write.

ఖరీదు చదివేవాడి మీదికి మారింది. అది ఎంత?

6. రెండో విరుపు – 52 రెట్లు queries, 96% వృథా

ఒక ముఖ్యమైన వివరం ఇక్కడ ఉంది, మరియు నేను మొదట దాన్ని మర్చిపోయాను: ఈ scale lo posts ఒకే database lo ఉండవు. అవి shards మీద పంచబడి ఉంటాయి.

నేను follow చేసే 400 మంది – వాళ్ళ posts 400 వేర్వేరు shards మీద ఉండొచ్చు.

నా మొదటి కొలత తప్పుగా వచ్చింది – ఎందుకంటే నేను SHARDING ని మర్చిపోయాను

మొదట నేను "100 followees ని ఒకే query lo తేవచ్చు" అనుకున్నాను. అప్పుడు fan-out on read కేవలం 1.2 రెట్లు ఖరీదు అని వచ్చింది – అంటే దాదాపు ఉచితం.

అది తప్పు. WHERE author IN (...) ఒకే machine మీద ఒక్క query; 256 shards మీద అది 256 queries.

Sharding ని model lo చేర్చక సంఖ్య 1.2 నుంచి 52 కి మారింది. నా model నిజం కంటే దయగా ఉంటే, అది నాకు అబద్ధం చెప్పింది.

posts 256 shards మీద • ఒక్క shard query 2 ms • ఒక్క author నుంచి 20 posts

follow చేస్తున్నవారు | తాకే shards | latency (సమాంతరంగా) | కలపాల్సిన posts

p50: 43	40	2 ms	860
p90: 107	88	4 ms	2,140
p99: 387	200	8 ms	7,740
గరిష్ఠం: 13,863	256	8 ms	2,77,260

మొత్తం భారం • 20,00,000 timeline reads

fan-out on read : 103 మిలియన్ shard queries (సగటు 52/read)

fan-out on write : 2 మిలియన్ queries (1/read)

తేడా : 52 రెట్లు

మరియు ప్రతి read lo సగటున 1,329 posts తెచ్చి, 50 కోసం sort చెయ్యాలి
అంటే తెచ్చిన దాంట్లో 3.8% మాత్రమే వాడతాం.

LATENCY బాగుంది • భారం ఫూరం – ఈ తేడా ముఖ్యం

Latency నిలుపు వరుస చూడండి: **2 నుంచి 8 ms**. అది అద్భుతం! ఎందుకంటే 256 queries **సమాంతరంగా** వెళ్తాయి.

కాబట్టి ఒక్క user ని చూస్తే ఈ design **వేగంగా** కనిపిస్తుంది. Load test lo ఒక user తో పరీక్షిస్తే అది **pass** అవుతుంది.

కానీ మొత్తం system మీద అది **52 రెట్లు** queries, మరియు **96.2% వృథా** – 1,329 posts తెచ్చి 50 వాడతాం.

Fan-out on read latency మీద విఫలం కాదు, ఖర్చు మీద విఫలమవుతుంది. మరియు ఖర్చు మీద విఫలమయ్యే designs చాలాకాలం బాగానే కనిపిస్తాయి – మీ bill వచ్చేవరకు.

Part 4 – మూడో విరుపు: ఎవరూ చదవని inbox

7. Step – hybrid · celebrities ని pull, మిగతా అందరినీ push

రెండు విరుపులూ ఒకే విషయం చెప్తున్నాయి: **ఒకే వ్యూహం అందరికీ సరిపోదు.**

- Fan-out on **write** చిన్నవాళ్ళకి అద్భుతం (93 writes), celebrities కి విపత్తు (1 కోటి writes)
- Fan-out on **read** celebrities కి అద్భుతం (1 write), అందరికీ ఖరీదు (52× queries)

కాబట్టి రెండూ వాడదాం:

```
post(author, text) {  
  ...  
  const pushed = !this.isCeleb(author);  
  if (!pushed) return id;           // celebrity → ఎవరికీ రాయం · §5 · pull  
  for (const f of this.#followers.get(author) || []) {  
    ...                             // మిగతా అందరికీ → push  
  }  
}
```

ఒక్క **if**. మరియు §1 సంఖ్యల ప్రకారం ఇది **47.5% పనిని** తీసేస్తుంది (పైన 1% వాళ్ళు సృష్టించేది).

ఇది సరైన జవాబు, మరియు ప్రతి interview lo ఇది చెప్పాలి. కానీ ఇది ఒక జవాబు కాదు – ఇది ఒక **కొత్త problem** యొక్క ప్రారంభం.

మొదటి సూచన ఇది: push చేసినదాన్ని ఎవరు చదువుతున్నారు?

8. మూడో విరుపు – రాసిన దాంట్లో 90% చదవబడదు

Push చేయడంలో ఒక మౌనమైన ఊహ ఉంది: **inbox lo పెట్టినదాన్ని ఎవరో ఒకరు చదువుతారు.**

కానీ users చురుకుదనం కూడా ఒక power law. చాలామంది ఖాతా తెరిచి మర్చిపోతారు.

రోజువారీ చురుకుదనం (DAU) తగ్గితే వృథా ఎంత పెరుగుతుంది?

DAU	inbox writes	చదవబడనివి	వృథా
45%	37 మి	20 మి	55%
30%	24 మి	17 మి	70%

20%	13 మి	10 మి	80%
10%	6 మి	5 మి	90%

DAU 10% దగ్గర - మీ FAN-OUT పనిలో 90% ఎవరూ చూడరు

మరియు 10% DAU అనేది అసాధారణం కాదు; చాలా social products అక్కడే ఉంటాయి.

వృథా సూత్రం సులభం: **వృథా = 1 - DAU**. మీ system యొక్క అతి పెద్ద ఖర్చు - inbox writes - **మీ product యొక్క engagement కి విలోమానుపాతంలో** ఉంటుంది.

అంటే: మీ product ఎంత తక్కువ వాడితే, మీ infrastructure bill అంత ఎక్కువ నిష్పత్తిలో వృథా అవుతుంది.

పరిష్కారం స్పష్టం: **చురుకుగా ఉన్నవాళ్ళకి మాత్రమే push చేయడం**. మిగతావాళ్ళు తిరిగి వచ్చినప్పుడు pull చేసుకోవచ్చు.

```
for (const f of this.#followers.get(author) || []) {
  if (!this.#isActive(f)) {
    // §11 - వృథా వద్దు
    ...
    this.stats.skippedInactive++;
    continue;
  }
  ...
}
```

ఒక్క `if`, మరియు 90% ఆదా.

కానీ ఇక్కడే ఈ doc యొక్క అసలు కథ మొదలవుతుంది. ఆ `continue` ఒక మౌనమైన వాగ్దానాన్ని విరగొట్టింది, మరియు దాన్ని సరిచేయడానికి నాకు ఏడు ప్రయత్నాలు పట్టింది.

Part 5 – పూర్తి system

9. Step – inbox ఒక cache, ఒక సత్యం కాదు

§7 మరియు §8 lo మనం రెండు సార్లు push ని దాటేశాం:

- Celebrity అయితే – ఎవరికీ రాయలేదు
- Follower చురుకుగా లేకపోతే – అతనికి రాయలేదు

ప్రతిసారీ ఆ inbox అసంపూర్ణం అయింది. మరియు అసంపూర్ణమైన inbox ని పూర్తి అనుకుని చదివితే, posts శాశ్వతంగా అదృశ్యమవుతాయి.

ఇదే ఈ problem యొక్క నిజమైన కేంద్రం, మరియు ఇది ఒక్క వాక్యంలో ఉంది:

ఈ DOC యొక్క కేంద్ర వాక్యం

Inbox ఒక సత్యం కాదు – అది posts table యొక్క ఒక cache.

మరియు ప్రతి cache కి ఒక చెల్లుబాటు షరతు కావాలి: "ఇది ఎప్పుడు నమ్మదగినది?"

§10 lo నేను ఆ షరతుని ఐదు సూక్ష్మ నియమాల కలయికగా రాయడానికి ప్రయత్నించాను, మరియు ఐదుసార్లు తప్పాను. ప్రతిసారీ ఒక రంధ్రం మూసి ఇంకొకటి తెరిచాను.

10. మొత్తం code · ఏడు ప్రయత్నాలు · 2 లక్షల timelines

నాలుగు నియమాలు

#	నియమం	విరిగితే అర్థం
1	Timeline సమయం ప్రకారం తగ్గుతూ ఉండాలి	కలపడంలో తప్పు
2	ఒకే post రెండుసార్లు రాకూడదు	push మరియు pull ఒకదాన్నొకటి నకలు చేస్తున్నాయి
3	Follow చెయ్యనివాళ్ళవి, తీసేసినవి రాకూడదు	వడపోత లేదు
4	ఉండాల్సిన top-N posts అన్నీ ఉండాలి (పూర్ణత)	ఒక post ఎక్కడో పోయింది

నియమం 4 ఒక్కటే ఈ doc మొత్తం. మిగతా మూడూ సులభంగా నెగ్గాయి. నాలుగోది ఏడుసార్లు విఫలమైంది.

ఏడు ప్రయత్నాలు

#	ఏమి తప్పు	ఎన్ని ప్రయోగాలు విఫలం
1	Inactive కి push దాటేశాను, కానీ వాళ్ళు తిరిగి వచ్చినప్పుడు పూడ్చలేదు	1,000 / 1,000
2	Watermark పెట్టాను, కానీ కొత్తగా follow చేసినవాళ్ళ పాత posts రాలేదు	1,000
3	Back-fill చేశాను, కానీ "celebrity" ని user లక్షణంగా చూశాను – follower సంఖ్య మారితే posts రెండ్రంలో పడతాయి	1,000
4	Push కాని posts కి ప్రత్యేక జాబితా చేశాను, కానీ pull చేసినదాన్ని inbox lo రాయలేదు	915
5	Pull ని inbox lo రాశాను, కానీ ఒక push watermark ని ముందుకి జరిపేస్తోంది – అంతకుముందు దాటేసినవి మరుగున పడ్డాయి	47
6	రెండ్రాన్ని గుర్తుపెట్టాను, కానీ inbox trim అయినప్పుడు ఆ గుర్తు లేదు	10
7	Trim ని గుర్తుపెట్టాను, కానీ deep pull ఫలితాలు inbox lo పడలేదు – జవాబు ఇచ్చి "శుభ్రం" అని గుర్తుపెట్టాను	2
✓	Deep pull ని కూడా inbox lo రాయడం	0

ఈ ఏడు వరుసలని మళ్ళీ చదవండి – వాటిలో ఒక ఆకారం ఉంది

ప్రయత్నాలు 1, 4, 5, 7 అన్నీ ఒకే తప్పు: "నేను ఈ post ని చూశాను" అనేదాన్ని "ఈ post inbox lo ఉంది" అనుకోవడం.

ప్రయత్నం 3 వేరే ఆకారం, మరియు అది నాకు బాగా నచ్చింది: "Celebrity" అనేది user లక్షణం కాదు – అది ఈ post యొక్క లక్షణం. Follower సంఖ్య మారుతూ ఉంటుంది; ఒక post push అయిందా లేదా అన్నది post పెట్టిన క్షణంలోనే తేలిపోతుంది, మరియు అది అక్కడే నమోదు కావాలి.

లేకపోతే: post పెట్టినప్పుడు celebrity (కాబట్టి push కాలేదు) → తర్వాత followers తగ్గారు (కాబట్టి ఇక pull కాదు) → ఆ post ఎవరికీ ఎప్పుటికీ కనిపించదు.

ఎనిమిదో ప్రయత్నం – షరతుల కలయికని వదిలేయడం

ఏడు ప్రయత్నాల తర్వాత నేను ఒక విషయం గమనించాను: నా "inbox నమ్మదగినదా?" అనే షరత ఐదు ప్రీడికేట్ల కలయిక అయిపోయింది – రెండ్రం, trim అంచు, అసంపూర్ణ repair, celebrity చరిత్ర, లోతు. ప్రతిసారీ ఒకటి మర్చిపోయాను.

కాబట్టి దాన్ని ఒకే ప్రశ్నగా మార్చాను:

```
// inbox ఒక *cache*. దాని చెల్లుబాటు షరతుని ఐదు సూక్ష్మ నియమాల కలయికగా
// రాయడానికి ప్రయత్నించి ఐదుసార్లు తప్పాను (§13). కాబట్టి ఇప్పుడు ఒకే ప్రశ్న:
// "ఈ inbox ని ఏదైనా అపవిత్రం చేసిందా?" – skip, trim, లేదా అసంపూర్ణ repair.
// అవును అయితే మూలం నుంచి ధృవీకరించు. ఎక్కువగా pull చేస్తుంది,
// కానీ ఎప్పుడూ తప్పు కాదు.
#dirty = new Set();
```

Skip అయినా, trim అయినా – ఆ user #dirty lo చేరతాడు. timeline() lo:

```
if (this.#dirty.has(user) || out.size < limit || touchesEdge || truncated) {
  this.stats.deepPull++;
  // ... మూలం నుంచి ధృవీకరించు, మరియు ఫలితాలని inbox lo కూడా రాయి
}
```

ఇదే అసలు పాఠం, మరియు ఇది **CACHE** ల గురించి సాధారణ సత్యం

ఒక cache యొక్క చెల్లుబాటు షరతుని మీరు ఒక్క వాక్యంలో చెప్పలేకపోతే, ఆ cache తప్పు.

నా ఐదు-ప్రిడిక్ట్ల షరత ప్రతినారీ "దాదాపు" సరైనది – 1,000 lo 998 సార్లు. మరియు అది సరిగ్గా అంత ప్రమాదకరమైనది: **అది పని చేస్తున్నట్టు కనిపిస్తుంది.**

"ఏదైనా అపవిత్రం చేసిందా?" అనేది ఒక్క వాక్యం, మరియు అది **సంప్రదాయవాదం** – అవసరమైనదాని కంటే ఎక్కువసార్లు pull చేస్తుంది. కానీ అది **ఎప్పుడూ తప్పు కాదు**, మరియు దాని ఖర్చు కొలవదగినది.

ఫలితం

1,000 యాదృచ్ఛిక ప్రయోగాలు · 1,67,279 posts · 2,04,393 timelines
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

posted	1,67,279
fanoutWrites	1,08,366
skippedCeleb	9,681
skippedInactive	1,39,372
pulled	44,085
trimmed	5,027
timelines	2,04,393
backfilled	4,04,035
repaired	1,90,111
deepPull	1,82,469
partialRepair	120

Test విఫలం కాగలదా? – ఎనిమిది మార్పులు

మార్పు లేని code → 0/1000 విఫలం

skip అయినప్పుడు రంధ్రం గుర్తుపెట్టకపోతే	→	1/300 విఫలం
skip అయినప్పుడు dirty గుర్తు పెట్టకపోతే	→	1/300 విఫలం
trim అయినప్పుడు dirty గుర్తు పెట్టకపోతే	→	0/300 విఫలం
deep pull ఫలితాలని inbox lo రాయకపోతే	→	1/300 విఫలం
follow చేసినప్పుడు వెనక్కి పూడ్చకపోతే	→	221/300 విఫలం
అసంపూర్ణ repair తర్వాత కూడా రంధ్రం తీసేస్తే	→	1/300 విఫలం
unfold తర్వాత inbox వడపోత తీసేస్తే	→	300/300 విఫలం
duplicate తీసేసే Map ని రెట్టింపు చేస్తే	→	300/300 విఫలం

1/300 - ఈ సంఖ్యని చూడండి

నాలుగు మార్పులు **300 ప్రయోగాల్లో ఒక్కసారే** పట్టుబడ్డాయి. అంటే ఆ bugs **0.3% సార్లు** కనిపిస్తాయి.

అందుకే నాకు ఏడు ప్రయత్నాలు పట్టింది. ప్రతిసారీ నా code **99.7% సరిగ్గా** ఉండేది - మరియు ఆ 0.3% lo ఒక user యొక్క ఒక post **శాశ్వతంగా** అదృశ్యమయ్యేది.

ఒక unit test ఈ పరిస్థితిని ఎప్పటికీ రాయదు: "ఒక user inactive గా ఉండగా ఒక post రావాలి, తర్వాత అతను active అవ్వాలి, తర్వాత ఇంకో push రావాలి, తర్వాత అతను timeline అడగాలి." ఆ మూడు అడుగుల కలయికని **ఎవరూ ఊహించరు**.

మరియు trim మార్పు **0/300** - ఎందుకంటే నా అమరికలో trim అరుదు (5,027 సార్లు). అది test యొక్క బలహీనత, code యొక్క బలం కాదు.

Part 6 – Interview Io

11. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-5	Followers పంపిణీ. p50 93, గరిష్ఠం 19 లక్షలు, సగటు ఒక అబద్ధం
5-9	§2 ప్రశ్నలు. ముఖ్యంగా "అత్యధిక followers ఎంత?"
9-17	మొదటి విరుపు. Fan-out on write రాసి, queue బొమ్మ గీసి, 56 ms → 199.9 s
17-25	రెండో విరుపు. Fan-out on read. Latency బాగుంది కానీ 52x భారం, 96% వృథా . Sharding ని మర్చిపోవద్దు
25-30	Hybrid – ఒక్క if , 47.5% పని ఆదా
30-36	మూడో విరుపు. DAU 10% అయితే 90% వృథా . Active కి మాత్రమే push
36-43	"కానీ ఇప్పుడు inbox అసంపూర్ణం." ఏడు ప్రయత్నాల కథ – కనీసం మూడు చెప్పండి
43-45	"Inbox ఒక cache; దాని చెల్లుబాటు షరతు ఒక్క వాక్యంలో ఉండాలి"

ఏమి తప్పక చెప్పాలి

- సగటు **followers ఒక అబద్ధం**. p50 93, గరిష్ఠం 19 లక్షలు. Design గరిష్ఠం నుంచి వస్తుంది.
- Celebrity problem** అతని సమస్య కాదు, అతని వెనక ఉన్నవాళ్ళ సమస్య. Head-of-line blocking.
- Fan-out on read latency** మీద విఫలం కాదు, ఖర్చు మీద విఫలమవుతుంది. అందుకే load test lo pass అవుతుంది.
- "Celebrity"** అనేది **post యొక్క లక్షణం, user లక్షణం కాదు**. ఇది చెప్తే మీరు దీన్ని నిజంగా రాశారని తెలుస్తుంది.
- Inbox ఒక cache**. Push దాటసిన ప్రతిసారీ అది అపవిత్రమవుతుంది, మరియు దాన్ని ధృవీకరించే దారి ఉండాలి.

ఏమి వదిలేయాలి

- Ranking / ML – అడిగితే మాత్రమే (§13)
- Inbox ని ఏ database lo పెట్టాలి – "Redis లేదా Cassandra" అని ఒక వాక్యం
- Media (photos/video) – వేరే problem
- Pagination యొక్క cursor రూపం

12. నీటితో చెప్పాల్సిన English script

"Before choosing a strategy I want one number: the **shape** of the follower distribution. In my model the median user has 93 followers, p99 has 3,057, and the maximum has 1.9 million. The mean is 349 – nearly four times the median – which is the signature of a power law and means **the average is a lie**. The top 0.1% of users generate 29.5% of all fan-out work."

"Fan-out on write is the obvious start: on post, write into every follower's inbox, so a read is one query no matter how many people you follow. At the median that's 93 writes, about two milliseconds. Fine."

"But fan-out is a queue, and a celebrity post is one enormous item in it. I measured a normal post appearing in **56 milliseconds**. With one ten-million-follower account posting, the hundred posts behind it waited an average of **199.9 seconds**. Those posts belong to ordinary users with 93 followers each – two milliseconds of work – and they waited three and a half minutes. **The celebrity problem isn't the celebrity's problem, it's everyone behind them**. And because celebrity posts are rare, p50 and p99 both look healthy; this never shows on your dashboard."

"The opposite approach – fan-out on read – fixes that completely. One write per post. But here I made a modelling mistake worth mentioning: I first assumed you could fetch a hundred authors in one query, which made read-fan-out look almost free at 1.2× the cost. At this scale posts are **sharded**, so those hundred authors are on a hundred different shards. With 256 shards, the median read touches 40 of them. The real number is **52× the queries**, and you fetch 1,329 posts to display 50 – **96% waste**. Latency still looks fine, 2 to 8 milliseconds, because the shard queries run in parallel. **Fan-out on read doesn't fail on latency, it fails on cost** – which is why it passes load tests and shows up on your bill."

"So: hybrid. Push for normal users, pull for celebrities. One **if**, and it removes 47.5% of the work. And pushing only to **active** users removes more – I measured the waste as exactly **1 – DAU**, so at a 10% daily-active rate, **90% of inbox writes are never read**."

"And that's where the real problem starts, because both of those skips break a promise. The inbox is no longer the truth – it's a **cache of the posts table**, and it's now missing things. I'll be honest about what happened when I built it: it took me **seven attempts** to get completeness right, and every attempt closed one hole and opened another. Missing back-fill when an inactive user returns. Missing older posts from someone you just followed. Then the one I liked most: I was treating **'celebrity' as a property of the user** when it's a property of the **post** – if someone is above the threshold when they post, the post is never pushed, and if their follower count later drops, the pull path stops covering them and that post is invisible forever."

"Then three more, all the same shape: **confusing 'I returned this post' with 'this post is in the inbox.'** Pulling a post into the response and then marking the inbox clean loses it on the next read."

"What finally worked wasn't a smarter predicate. My validity condition had become a conjunction of five subtle checks and I kept forgetting one – it was right 998 times in 1,000, which is exactly dangerous enough to look correct. I replaced it with one question: **has anything invalidated this inbox** – a skip, a trim, a partial repair? If so, verify against the source. That pulls more often than strictly necessary, but it's never wrong, and the extra cost is measurable."

"Four invariants, thousand randomized runs, 167,000 posts, 204,000 timelines, zero violations. Worth noting: four of the eight mutations are caught in only **1 of 300 runs** – those bugs appear 0.3% of the time, which is why they took seven attempts. No unit test would ever write the sequence 'user goes inactive, post arrives, user becomes active, another post arrives, user reads.'"

13. Follow-ups – ranking, stories, unfollow

"Timeline కాలక్రమంలో కాకుండా ranked అయితే?"

అప్పుడు inbox lo post id తో పాటు **ranking features** కూడా ఉండాలి, లేదా చదివేటప్పుడు వాటిని తేవాలి. ముఖ్యమైన మార్పు: **ranked timeline lo "పూర్ణత" అనే నియమం మారుతుంది.** కాలక్రమంలో top-50 అంటే ఒక ఖచ్చితమైన సమితి; ranked lo అది model మీద ఆధారపడుతుంది. అంటే \$10 యొక్క నియమం 4 ని పరీక్షించలేరు – కాబట్టి **ranking కంటే ముందు ఒక కాలక్రమ పాఠ** ఉండాలి, మరియు ranking దాని మీద పని చేయాలి. అప్పుడు పూర్ణతని ఇంకా పరీక్షించగలరు.

"Post delete అయితే కోట్ల inbox ప్రతులు ఏమవుతాయి?"

వాటిని **తీసేయకూడదు** – అది మళ్ళీ fan-out, మరియు §4 lo చూసిన అదే head-of-line blocking. బదులుగా **చదివేటప్పుడు వడపోయాలి** (if (p.deleted) continue). Inbox lo ఒక చనిపోయిన id ఉండటం ఖర్చు దాదాపు సున్నా. అదే తర్కం **unfollow** కి కూడా – §10 lo if (!follows.has(p.author)) continue .

ఒక సూక్ష్మమైన విషయం: తీసేసిన posts scan బడ్జెట్‌ని తినకూడదు. నా code lo మొదట **n++** తీసేసినవాటికీ లెక్కయ్యేది – కాబట్టి ఒక author కి చాలా deleted posts ఉంటే, వాటి కింద ఉన్న సజీవమైనవి ఎప్పటికీ కనిపించేవి కావు.

"Inbox ఎంత పెద్దది ఉండాలి?"

అది మీ timeline యొక్క **గరిష్ట లోతు** ని నిర్ణయిస్తుంది. నా code lo **inboxCap** దాటితే పాతవి trim అవుతాయి – మరియు §10 ప్రయత్నం 6 lo నేర్చుకున్నట్లు, **ఆ trim ఒక రంధ్రం**. Trim అయిన user inbox ఇక పూర్తి కాదు, కాబట్టి లోతైన pagination **తప్పకుండా** మూలం నుంచి రావాలి. నిజమైన systems ఇదే చేస్తాయి: మొదటి పేజీ inbox నుంచి, లోతైన పేజీలు posts table నుంచి.

"ఒక user 5,000 మందిని follow చేస్తే?"

§6 పట్టిక ప్రకారం అతని deep pull **256 shards** తాకుతుంది. అలాంటివాళ్ళు అరుదు (p99 కూడా 387), కాబట్టి వాళ్ళకి ప్రత్యేకంగా **inbox ని ఎప్పుడూ push** చేయడం విలువైనది – అంటే వాళ్ళని ఎప్పుడూ "active" గా పరిగణించడం. ఇది §8 యొక్క విలోమం, మరియు అది సరైనదే: **ఎవరికి push చేయాలో నిర్ణయించేది వాళ్ళ చురుకుదనం మాత్రమే కాదు, వాళ్ళ pull ఖర్చు కూడా.**

14. ఏమి నేర్చుకున్నాం

1. సగటు ఒక అబద్ధం. p50 93 followers, గరిష్టం 19 లక్షలు – 20,000 రెట్లు తేడా. మీ design సగటు నుంచి కాదు, **తేక నుంచి** రావాలి. పైన 0.1% users 29.5% పని సృష్టిస్తారు.

2. ఒక పెద్ద పని queue lo ఉంటే, అది అందరిదీ సమస్య. Celebrity post వెనక ఉన్న సాధారణ posts **56 ms నుంచి 199.9 సెకన్లు** ఆగాయి. మరియు celebrity posts అరుదు కాబట్టి **p50, p99 రెండూ ఆరేగ్యంగా** కనిపిస్తాయి.

3. Latency మరియు ఖర్చు వేర్వేరు వైఫల్యాలు. Fan-out on read **2-8 ms** – అద్భుతం. అదే సమయంలో **52x queries మరియు 96% వృథా**. Latency test pass అవుతుంది, bill fail అవుతుంది.

4. నా model నిజం కంటే దయగా ఉంటే అది నాకు అబద్ధం చెప్తుంది. Sharding ని మర్చిపోయినప్పుడు fan-out on read **1.2 రెట్లు** ఖరీదు అని వచ్చింది. దాన్ని చేర్చాక **52 రెట్లు**.

5. వృథా = 1 – DAU. మీ product engagement తక్కువైతే, మీ fan-out ఖర్చులో వృథా నిప్పత్తి పెరుగుతుంది. 10% DAU దగ్గర 90%.

6. "Celebrity" అనేది user లక్షణం కాదు, post లక్షణం. ఆ నిర్ణయం post పెట్టిన క్షణంలో తీసుకుంటాం – కాబట్టి అది **అక్కడే నమోదు కావాలి**. లేకపోతే follower సంఖ్య మారినప్పుడు posts రెండు వ్యూహాల మధ్య రంధ్రంలో పడతాయి.

7. "చూశాను" అంటే "inbox lo ఉంది" కాదు. నా ఏడు ప్రయత్నాల్లో నాలుగు ఇదే తప్పు. Pull చేసి జవాబు ఇచ్చి, inbox ని "శుభ్రం" అని గుర్తుపెడితే – తర్వాతి read lo అదే post మళ్ళీ పోతుంది.

8. Cache యొక్క చెల్లుబాటు షరతుని ఒక్క వాక్యంలో చెప్పలేకపోతే, ఆ cache తప్పు. నా ఐదు-ప్రీడిక్ట్డ్ షరతు 1,000 lo 998 సార్లు సరైనది – మరియు అదే దాన్ని ప్రమాదకరంగా చేసింది. ఒకే ప్రశ్నకి మార్చాక: 0 ఉల్లంఘనలు.

9. 0.3% సార్లు కనిపించే bug ఒక చిన్న bug కాదు. నాలుగు మార్పులు **300 lo ఒక్కసారే** పట్టుబడ్డాయి. అవి staging lo ఎప్పటికీ కనిపించవు – అవి ఒక user కి, ఒకసారి, ఒక post శాశ్వతంగా పోగొట్టి కనిపిస్తాయి.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

"Hybrid వాడండి" అనేది design యొక్క ముగింపు కాదు – అది ప్రారంభం.

Push మరియు pull ని కలపడం సులభం. కష్టమైనది – **ఆ రెండింటి మధ్య ఏదో పడిపోలేదని నిర్ధారించడం.**

ప్రతి "దీన్ని దాటేద్దాం" (celebrity, inactive, trim) ఒక రంధ్రం తెరుస్తుంది. మరియు ఆ రంధ్రాలు **0.3% సార్లు** కనిపిస్తాయి – అంటే మీ code పని చేస్తున్నట్లే కనిపిస్తూ, ఎవరో ఒకరి posts ని నిశ్శబ్దంగా మింగేస్తూ ఉంటుంది.